

MSBA503 YELP BIG DATA PIPELINE

Databricks Pipeline Parameter Guide

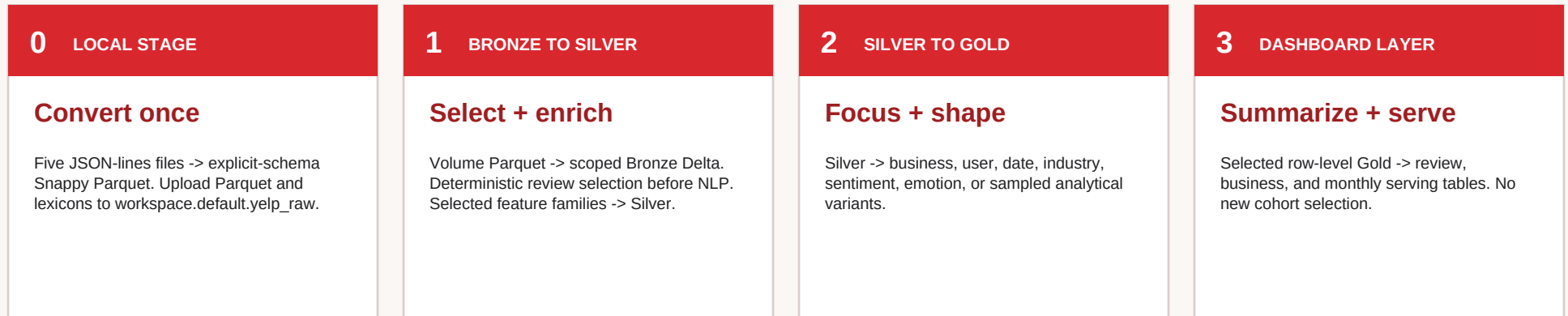
A visual operating manual for converting the Yelp Academic Dataset locally, staging governed Parquet in Unity Catalog, and producing Silver intelligence, flexible Gold cohorts, and dashboard-ready tables.

LOCAL JSON	PARQUET VOLUME	BRONZE	SILVER	GOLD	DASHBOARD
------------	----------------	--------	--------	------	-----------

ONE LOCAL STAGE. ONE DATABRICKS JOB. THREE DEPENDENT TASKS.

The pipeline at a glance

The 2026 workflow stages source bytes once, then runs one ordered Databricks Job. Each task owns one layer and passes a validated Delta table to the next task.



The dependency chain is the product

Silver contains the selected reviews with full requested feature engineering. Gold decides what analytical slice or grain is published. The dashboard never bypasses Gold. Local Parquet staging is reusable input infrastructure, not a fourth Job task.

Create the Volume hierarchy + upload

Catalog Explorer supports directories inside a Volume. Create the hierarchy first, then drag/drop each WSL output folder into its matching governed destination.

1 Convert the five JSON files locally

Use pipeline/bronze_json_to_parquet.py. Five _SUCCESS markers prove conversion completed.

2 Create workspace.default.yelp_raw

Open Catalog Explorer > workspace > default, create the Volume, then open its Files tab.

3 Create bronze and lexicons

Use Create directory twice at the Volume root. Do not upload loose files at the root.

4 Create five directories inside bronze

Use the exact business, checkin, review, tip, and user dataset directory names.

5 Open the WSL output in Windows Explorer

Browse to \\wsl.localhost\Ubuntu\home\yelp_parquet.

6 Drag/drop each Parquet group

Open one WSL dataset folder at a time and drop only part-*.snappy.parquet into its match.

7 Drag/drop the five lexicons

Place the exact .txt files in lexicons/. Skip .crc; _SUCCESS is optional.

Required Volume layout

```
/Volumes/workspace/default/yelp_raw/
bronze/yelp_academic_dataset_review/
bronze/yelp_academic_dataset_business/
bronze/yelp_academic_dataset_user/
bronze/yelp_academic_dataset_checkin/
bronze/yelp_academic_dataset_tip/
```

lexicons/ [five exact .txt files]

Expected upload

Review 64 | User 32 | Tip 8 | Business 4 | Check-in 4
112 Parquet parts + 5 lexicons. _SUCCESS optional; never upload .crc.

Job parameters + JSON grammar

Job parameters are reusable defaults. Each Python-script task still receives a JSON array of strings in its own Parameters textbox.

Name	Default	Used by
input_volume	/Volumes/workspace/default/yelp_raw/bronze	Bronze to Silver
lexicons_dir	/Volumes/workspace/default/yelp_raw/lexicons	Bronze to Silver
silver_sample_size	25000	Bronze to Silver
silver_sample_seed	222	Bronze to Silver
gold_sample_size	5000	Silver to Gold
gold_sample_seed	222	Silver to Gold
materialization_mode	auto	Bronze + Gold

Task Parameters JSON

```
[
  "--input-volume",
  "{{job.parameters.input_volume}}",
  "--lexicons-dir",
  "{{job.parameters.lexicons_dir}}",
  "--silver-sample-size", "25000",
  "--nlp-component", "linguistic",
  "--nlp-component", "vader"
]
```

JSON rules

Every flag and value is a separate quoted item. Quote numbers. Repeat the entire flag/value pair for repeatable options. Use `{{job.parameters.name}}` only when that Job parameter exists. Notebook tasks use a different key/value interface.

Two independent sampling controls

One limit controls what reaches Silver and incurs NLP work. The other controls the row-level Gold dataset presented to analysis and the dashboard.

BEFORE SILVER - sample the workload

--silver-sample-size N --silver-sample-seed S

Selects N review IDs deterministically before joins and feature engineering. Related business, user, check-in, and tip rows are scoped to those reviews. Only selected reviews become Silver.

AFTER GOLD FILTERS - shape the output

--gold-sample-size N --gold-sample-seed S

Applies after business, user, industry, date, geography, star, sentiment, and emotion filters. For brand-sample, N is applied separately to each selected brand.

```
effective_gold_sample_size = min(requested_gold_sample_size, eligible_silver_rows)
```

Safe convergence

An oversized Gold request does not fail and cannot invent rows. The task uses the eligible Silver population, emits a warning, and records both the requested and effective limits in the audit record.

RAW / BRONZE	SILVER	GOLD	DASHBOARD
review sample + related entities	fully featured selected reviews	chosen sample or aggregate	serving views from Gold

Bronze-to-Silver reference 1/3

Governed inputs, destination, execution, materialization, and workload controls for `bronze_to_silver_yelp`.

Purpose	Databricks Parameters JSON	Default / requirement
Help	"--help"	Optional
Bronze Parquet root	"--input-volume", "/Volumes/workspace/default/yelp_raw/bronze"	Required maintained source
Licensed lexicon directory	"--lexicons-dir", "/Volumes/workspace/default/yelp_raw/lexicons"	Required standard features
Destination catalog	"--catalog", "workspace"	workspace
Destination schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Delta write mode	"--mode", "overwrite"	overwrite errorifexists
Spark partitions	"--partitions", "32"	Optional
Arrow batch ceiling	"--arrow-batch-size", "5000"	5000 records
Materialization mode	"--materialization-mode", "auto"	auto persist delta none
Scratch schema	"--materialization-schema", "workspace.default"	workspace.default
Silver review limit	"--silver-sample-size", "25000"	All reviews if omitted
Silver seed	"--silver-sample-seed", "222"	222

Serverless materialization

auto selects temporary managed Delta tables under Spark Connect/serverless and persist/unpersist on classic compute. Temporary tables are recorded and cleaned on success or failure.

Bronze-to-Silver reference 2/3

Select full-corpus feature families by repeating `--nlp-component`. Omit it to run every standard group except transformers.

Feature group	Databricks Parameters JSON	What it adds
linguistic	"-nlp-component", "linguistic"	spaCy linguistic + structural features
vader	"-nlp-component", "vader"	VADER sentiment scores + labels
nrc-emotion	"-nlp-component", "nrc-emotion"	NRC emotion features
vad	"-nlp-component", "vad"	Valence, arousal, dominance
entities	"-nlp-component", "entities"	Named-entity features
grammar	"-nlp-component", "grammar"	Grammar + text-quality features
domain-lexicons	"-nlp-component", "domain-lexicons"	WorryWords, WCST, Yelp-domain features
time-weighting	"-nlp-component", "time-weighting"	Time-weighted analytical features
transformers	"-nlp-component", "transformers"	Optional local-model inference on compute

How pandas scales here

Spark partitions the review data and streams bounded Arrow batches into `mapInPandas`. pandas never receives the full dataset; peak pandas memory is governed by the Arrow batch ceiling on each executor.

Transformer boundary

Selecting transformers requires the model libraries and suitable compute on the task. It is not an API call and is deliberately excluded from the default feature set.

Bronze-to-Silver reference 3/3

Licensed lexicons are staged once in the governed Volume. The loader resolves exact filenames and serializes the five small immutable payloads to serverless workers.

Purpose	Databricks Parameters JSON / exact filename	Requirement
Volume directory	"--lexicons-dir", "/Volumes/workspace/default/yelp_raw/lexicons"	Required maintained path
NRC VAD	NRC-VAD-Lexicon-v2.1.txt	Required for VAD
WorryWords	worrywords-v1.txt	Domain features
NRC WCST	NRC-WCST-Lexicon-v1.0.txt	Domain features
Yelp AFFLEX/NEGLEX	Yelp-restaurant-reviews-AFFLEX-NEGLEX-unigrams.txt	Yelp-domain features
NRC emotion intensity	NRC-Emotion-Intensity-Lexicon-v1.txt	Emotion intensity

Literal terms stay literal

The loaders disable pandas NA-word conversion, so vocabulary such as the English word 'null' remains a real term.

Upload rule

Upload only the five .txt files. Do not rename them, wrap them in another folder, or upload local .crc checksum files.

Silver-to-Gold reference 1/3

Input, output, strategy, and deterministic row-sampling controls for silver_to_gold_yelp.

Purpose	Databricks Parameters JSON	Default / requirement
Help	"--help"	Optional
Catalog	"--catalog", "workspace"	workspace
Schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Explicit Silver input	"--input-table", "CATALOG.SCHEMA.yelp_silver_reviews"	Derived when omitted
Explicit Gold output	"--output-table", "CATALOG.SCHEMA.OUTPUT"	Derived when omitted
Gold strategy	"--gold-level", "brand-sample"	Required
Gold row limit	"--gold-sample-size", "5000"	Required for row-level sample
Gold seed	"--gold-sample-seed", "222"	222

Compatibility aliases

--sample-size and --sample-seed remain accepted. New Job definitions should use the explicit Gold names.

Silver-to-Gold reference 2/3

Choose one Gold grain, then repeat any cohort filters needed for that run.

GOLD LEVELS

Level	Output
brand-sample	One deterministic row-level review sample per selected brand
sample	One deterministic row-level review sample after all filters
business	Business-level analytical aggregates
user	User-level analytical aggregates
date	Time aggregates at the selected date granularity
industry	Industry-level analytical aggregates
sentiment	Sentiment-band analytical aggregates
emotion	Emotion-label analytical aggregates

REPEATABLE FILTERS

Purpose	Databricks Parameters JSON
Brands	"--brand", "Chipotle", "--brand", "Great Clips"
Business IDs	"--business-id", "ID_1", "--business-id", "ID_2"
Business names	"--business-name", "NAME_1", "--business-name", "NAME_2"
User IDs	"--user-id", "ID_1", "--user-id", "ID_2"
Industries	"--industry", "Restaurants", "--industry", "Beauty & Spas"
Cities / states	"--city", "Phoenix", "--state", "AZ"
Sentiment labels	"--sentiment-label", "positive"
Emotion labels	"--emotion-label", "joy"

Dashboard compatibility

brand-sample and sample preserve review rows and can feed the dashboard task. Business, user, date, industry, sentiment, and emotion are analytical aggregates and are not interchangeable with the dashboard source.

Silver-to-Gold reference 3/3

Date, star, sentiment, and emotion controls narrow the eligible Silver population before row sampling or aggregation.

Purpose	Databricks Parameters JSON	Default / choices
Inclusive start date	"--start-date", "2018-01-01"	Optional ISO date
Inclusive end date	"--end-date", "2021-12-31"	Optional ISO date
Date aggregation	"--date-granularity", "month"	day week month quarter year
Minimum stars	"--min-stars", "1"	Optional
Maximum stars	"--max-stars", "5"	Optional
Sentiment feature	"--sentiment-column", "vader_sentiment_score"	VADER, HF, Yelp lexicon, or Valence
Minimum sentiment	"--min-sentiment", "-0.25"	Optional
Maximum sentiment	"--max-sentiment", "1.0"	Optional
Emotion feature	"--emotion-column", "dominant_emotion"	dominant, lexicon, or HF label

Case study is configuration

The template selects Chipotle and Great Clips for the coursework case study. The task itself is reusable: replace the brands or select businesses, users, industries, geography, time windows, sentiment bands, or emotion labels.

Data-science dashboard layer

The third task does not choose another cohort. It translates the selected row-level Gold table into stable serving tables for presentation and analysis.

Purpose	Databricks Parameters JSON	Default / requirement
Catalog	"--catalog", "workspace"	workspace
Schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Selected Gold table	"--gold-table", "workspace.default.yelp_gold_dashboard_variant"	Use row-level Gold

REVIEW SAMPLE	BUSINESS SUMMARY	MONTHLY SUMMARY
Row-level detail Table: yelp_dashboard_review_sample Review text, stars, features, and Gold variant	Operational rollup Table: yelp_dashboard_business_summary Review counts, sentiment, risk, and attention tier	Trend layer Table: yelp_dashboard_monthly_summary Monthly review, rating, and sentiment movement

Direct dependency

Whatever Gold selects is what the dashboard represents. Change the brands, filters, dates, industries, or Gold sample size; then rerun the dependent dashboard task to refresh the serving layer.

Implementation architecture

The 2026 revision turns the coursework feature work into a reproducible distributed pipeline without publishing or reading the repository's local data directory.

Capability	Implemented design
Source stage	Local explicit-schema JSON-lines -> Snappy Parquet; complete outputs carry <code>_SUCCESS</code>
Governed inputs	<code>workspace.default.yelp_raw</code> Volume with separate bronze/ and lexicons/ paths
Schemas	Explicit definitions for all five Yelp datasets
Feature path	One canonical pandas implementation distributed through Spark <code>mapInPandas</code>
Writes	Spark-native Parquet, CSV, and Delta; no full driver-side collection
Validation	Mandatory at-scale validation before featured Silver publication
Materialization	Persist on classic compute; temporary managed Delta on serverless/Spark Connect
Audit	Failure-path logging plus queryable Delta audit records
Lexicons	Volume bytes -> driver files -> immutable serverless worker payloads; SparkFiles on EMR
Cloud shape	Databricks serverless + transient EMR medallion pipelines
Presentation	Databricks dashboard-serving tables and Streamlit app layer

Production principle

Small pandas batches are an executor API, not a dataset destination. Spark owns partitioning, joins, materialization, validation, and distributed writes; pandas owns the canonical feature logic inside bounded Arrow batches.

Local release gate before cloud spend

The maintained cloud pipelines run on Databricks and EMR. This local Linux/WSL2 workflow is a verification gate for shared feature code, dependencies, Spark writes, and package build.

RECOMMENDED: WSL2 / LINUX

```
git clone <this repo> ~/yelp_iter2
cd ~/yelp_iter2
./scripts/bootstrap_local.sh
```

Clone inside the Linux filesystem under ~/ - never below /mnt/c. The bootstrap requires Python 3.11 and Java 17 exactly, creates .venv, installs the pinned requirements and spaCy model, downloads the two required NLTK resources, validates dependencies, and runs the strict suite.

RERUN WITHOUT REBUILDING

```
PYSPARK_PYTHON="$PWD/.venv/bin/python" \
PYSPARK_DRIVER_PYTHON="$PWD/.venv/bin/python" \
./venv/bin/python ./scripts/run_test_suite.py
```

NATIVE WINDOWS: CONDITIONAL

```
powershell -ExecutionPolicy Bypass \
-File .\scripts\bootstrap_local.ps1
```

Use a short checkout path without parentheses, such as C:\dev\yelp_iter2. PySpark 3.5 launches through cmd.exe batch files; parentheses can surface as the misleading JAVA_GATEWAY_EXITED error.

Real local Parquet-write tests also require a trusted Hadoop build with %HADOOP_HOME%\bin\winutils.exe. The bootstrap does not download an unofficial executable. Use WSL2/Linux when no verified Windows build is already available.

Genuine pass

GENUINE PASS: N tests executed; 0 failures, 0 errors, 0 skips. Any skip, zero-test run, dependency drift, version mismatch, or missing NLP resource fails the gate.

Deterministic EMR package evidence

The 2026-09-22 gate built SHA-256
a2d71d96a0ec8bb1b5113bc3c3c59f1327935e373893a45ee9200c2dd738ccd8.

Verification record

The strict local gate verifies the shared feature path before cloud spend. It is evidence for release readiness, not a substitute for a live cloud canary.

27

CURRENT TESTS

0

FAILURES

0

ERRORS

0

SKIPS

Current strict-suite contract

The current repository suite contains 27 tests. In the pinned Python 3.11 + Java 17 environment, a genuine pass requires all 27 to execute with 0 failures, 0 errors, and 0 skips. Any skip, zero-test run, missing dependency, drifted version, or missing NLP resource is rejected.

End-to-end parity - 2026-09-03

Two disjoint 1,500-row coursework slices each matched local and distributed results across all 68 expected feature columns, including schema, keys, nulls, strings, numerics, and row counts.

Evidence boundary

The suite uses synthetic temporary fixtures and touches no corpus or cloud account. Passing permits a small canary; it does not prove the full 6,990,280-review run. Cloud readiness becomes execution evidence only after a successful live audit record.

Release path + rules to remember

Local conversion and the free verification gate should precede the serverless canary; the canary should precede a larger sample.

1 Run the strict release gate

Accept only GENUINE PASS with 0 failures, 0 errors, and 0 skips.

2 Convert raw JSON locally

Require five _SUCCESS markers; completed datasets are restart-safe.

3 Verify the governed Volume

Preserve five Bronze dataset folders and five exact lexicon filenames.

4 Start with a small Databricks canary

Use the same environment, scripts, input paths, and sample controls planned for scale.

5 Inspect audit and runtime evidence

Check counts, validation, components, requested/effective samples, throughput, and failures.

6 Scale only after the canary

Increase limits deliberately while retaining the same audit and validation gates.

Rule	Meaning
DATABRICKS JSON	One array of quoted strings per Python-script task
INPUT VOLUME	Locally converted Bronze Parquet; never Git
SILVER LIMIT	Reviews that receive joins and NLP
GOLD LIMIT	Eligible row-level output after filters
LEXICONS	Five exact .txt files in the Volume
TRANSFORMERS	Optional compute dependency, not an API stage
DASHBOARD	Always depends on selected row-level Gold

Verify the installation

```
./venv/bin/python ./scripts/run_test_suite.py
Accept only: GENUINE PASS ... 0 failures, 0 errors, 0 skips.
```

Two maintained cloud routes

The same feature and analytical contracts are deployed through provider-native execution models. The choice is architectural, not a claim that the platforms are identical.

2026 PRIMARY ROUTE

Databricks serverless

Runtime

Serverless Spark Connect with no user-managed cluster.

Storage

Unity Catalog Volume inputs and governed Delta tables.

Materialization

Temporary managed Delta tables for reusable plans.

Job

Bronze-to-Silver -> Silver-to-Gold -> dashboard-serving tables.

2026 MAINTAINED ALTERNATIVE

AWS EMR on EC2

Runtime

Transient driver + workers on classic Spark/YARN.

Storage

S3 Parquet with AWS Glue registrations and Athena access.

Materialization

Classic persist/unpersist during the ordered workload.

Lifecycle

Terminates after success or on the first failed step.

Why transient EMR?

EMR also supports serverless execution. This repository uses a transient EC2-backed cluster to preserve the original MSBA 503 methodology while extending it into a complete Bronze, Silver, Gold, and dashboard-serving medallion pipeline.

One feature contract, provider-native execution

Both routes use the same canonical Python feature implementation and the same Gold and dashboard transformation contracts. Their storage, catalog, materialization, packaging, and lifecycle controls remain provider-specific so that each deployment is operationally honest.

AWS administrator foundation

The administrator creates the account-level storage, network, and IAM boundary. The pipeline user receives named resources and scoped launch permission - never blanket AdministratorAccess.

CREATE ONE PRIVATE, ENCRYPTED S3 PROJECT ROOT

```
s3://BUCKET/yelp/
|-- raw/                [upload 5 JSON files]
|-- lexicons/           [upload 5 .txt files]
|-- bootstrap/bootstrap.sh [upload]
|-- pipeline/
|   |-- bronze_json_to_parquet.py
|   |-- pipeline_modules.zip
|-- tasks/              [upload 3 task scripts]
|-- bronze-source/      [job creates]
|-- lakehouse/
|   |-- bronze/ |-- silver/ |-- gold/
|   |-- dashboard/      [job creates]
|-- audit/              [job creates]
|-- emr-logs/           [EMR creates]
```

Keep Block Public Access enabled. Put the bucket in the cluster Region. If a customer-managed KMS key is used, both the administrator and cluster role need the corresponding key permissions.

ADMINISTRATOR-OWNED PREREQUISITES

- 1 **Region + quota**
Confirm EMR 7.13, selected EC2 types, service quota, and capacity in one Region.
- 2 **VPC + subnet**
Provide a subnet with S3 access. A private subnet needs an S3 gateway endpoint or an approved NAT path.
- 3 **EMR service role**
Create or approve an EMR service role based on AmazonEMRServicePolicy_v2 and its required resource-tag controls.
- 4 **EC2 instance profile**
Create a custom cluster EC2 role and an instance profile with the same name. Trust ec2.amazonaws.com.
- 5 **Catalog access**
Authorize the cluster role for the project Glue catalog/database/tables and any Lake Formation boundary in use.

Least-privilege IAM boundary

EMR service role

Provision and manage EMR/EC2 resources. Trust: elasticmapreduce.amazonaws.com.

Cluster EC2 role

S3 List/Get/Put/Delete on BUCKET/yelp/*; Glue database/table read-write actions; KMS only if used.

Pipeline user / launcher

RunJobFlow plus view/terminate actions and iam:PassRole limited to the two approved roles and services.

AWS references: EMR IAM roles, custom roles, private-subnet S3 access, and Glue metastore permissions in the Amazon EMR documentation.

Package, parameterize, launch, verify

The repository launcher submits one complete RunJobFlow document. The four ordered steps share one transient cluster and stop the cluster automatically after success or on the first failure.

1 Build

Run `build_pipeline_package.py` after every pipeline-code change.

2 Upload

Place the JSON, lexicons, bootstrap, ZIP, converter, and three task scripts at the exact S3 paths.

3 Configure

Copy the template locally and replace bucket, subnet, role, driver, and worker placeholders.

4 Canary

Add a small Silver limit, keep termination controls enabled, launch, then inspect logs, audit, and Glue.

PARAMETER FORMAT: JSON TEMPLATE, TOKENIZED STEP ARGUMENTS

Unlike Databricks, EMR has no separate task Parameters textbox in this workflow. Edit `emr_cluster_config.json` (or a private canary copy). The full file is JSON, and every flag and value is a separate quoted string in HadoopJarStep.Args. Numbers are quoted because Args is a string list; repeat the full flag/value pair for repeatable options.

```
"Args": [
  "spark-submit",
  "--py-files", "s3://BUCKET/yelp/pipeline/pipeline_modules.zip",
  "s3://BUCKET/yelp/tasks/bronze_to_silver.py",
  "--silver-sample-size", "250",
  "--nlp-component", "linguistic",
  "--nlp-component", "vader"
]
```

CANARY BEFORE SCALE

Start with `--silver-sample-size 250` and modest partitions. Keep `KeepJobFlowAliveWhenNoSteps=false` and every `ActionOnFailure=TERMINATE_CLUSTER`. After the canary passes, remove the Silver limit and resize workers or partitions from measured evidence. EMR output is durable in S3; Streamlit should be hosted separately and query the serving layer through Athena.

Launch from AWS CloudShell

```
python aws_emr/launch_cluster.py \
  --config CONFIG --region REGION
```

- No `REPLACE_WITH_` placeholders remain.
- Four EMR steps finish in order.
- S3 audit reports success and row counts.
- Glue tables query from Athena.
- Cluster state reaches `TERMINATED`.